设备 MQTT 接入协议指南

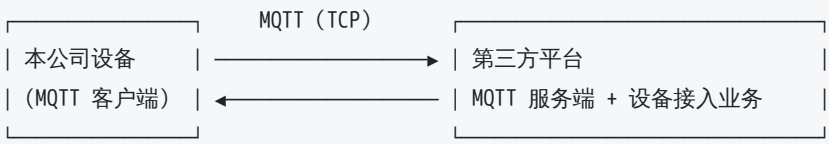
项目	内容
文档版本	V1.0
发布日期	2026-10-26
适用范围	本公司所有支持 MQTT 接入的设备（直连设备与网关设备）
读者对象	需要将本公司设备接入自有物联网平台的第三方平台技术人员

> 本文档说明本公司设备作为 MQTT 客户端时的连接方式、Topic、报文格式和交互时序，以及平台为完成对接需要提供和实现的内容。各型号设备上报的具体数据点，请参阅对应设备的数据协议文档。

目录

1. 对接概述
2. MQTT 协议简介
3. 平台需提供的条件
4. 设备端可配置参数
5. 连接与认证
6. Topic 定义
7. 遥测数据
8. 属性
9. 网关设备
10. 设备自动注册
11. 设备上线时序与连接行为
12. 对接自检清单
13. 常见问题排查
14. 附录

1. 对接概述



- 设备是 MQTT 客户端，主动连接平台提供的 MQTT 服务端。
- 平台负责：认证设备、接收并解析设备上报的数据、向设备推送配置、响应设备的查询请求。
- 设备与平台交换两类数据：

类别	方向	含义	示例
遥测	设备 → 平台	随时间变化的测量值，平台应按时间序列存储	温度、电压、运行状态
属性	双向	相对静态的信息，平台只需保存最新值	设备上报：固件版本、序列号；平台下发：上报周期、阈值

属性分为两种，在报文中用不同字段名区分：

属性类型	写入方	报文字段名
设备属性	设备上报，平台保存	`client` / `clientKeys`
配置属性	平台写入并推送给设备，设备也会主动查询	`shared` / `sharedKeys`

2. MQTT 协议简介

MQTT 是 OASIS 标准的轻量级发布/订阅消息协议，运行在 TCP 之上。客户端连接服务端后，通过 Topic（以 `/` 分隔的层级字符串）发布（PUBLISH） 或 订阅（SUBSCRIBE） 消息，服务端负责按 Topic 转发。

概念	说明
通配符 `+` / `#`	仅用于订阅。`+` 匹配一个层级，`#` 匹配其后所有层级
QoS	0：至多一次；1：至少一次（可能重复）；2：恰好一次。设备使用 QoS 1
Keep Alive	心跳间隔；服务端在 1.5 倍该时间内未收到任何报文即可判定连接断开
Clean Session	设备使用 `true`，每次连接都是新会话，并在连接后重新订阅
Client ID	客户端标识；服务端对相同 Client ID 的新连接应踢掉旧连接
Retain / Will	设备不使用

3. 平台需提供的条件

项目	要求
MQTT 服务端	支持 MQTT 3.1.1（设备缺省版本），可选支持 3.1 / 5.0；支持 QoS 0 与 QoS 1
网络	服务端地址与端口对设备可达（缺省端口 `1883`）
设备凭据	为每台设备分配访问令牌，或用户名 / 密码（见第 5 章）
报文大小	单条报文上限不小于 64 KB
订阅授权	允许设备订阅第 6 章所列的下行 Topic
业务处理	实现第 7 ~ 10 章中平台侧的处理逻辑

平台需将以下信息提供给设备安装 / 配置人员，用于设置设备端参数（第 4 章）：服务端地址、端口、每台设备的凭据，以及平台若使用了自定义 Topic，则提供对应的 Topic 列表。

4. 设备端可配置参数

以下参数在设备现场安装时设置，"缺省值"为设备出厂值。平台对接时请确认这些参数与平台配置一致。

4.1 连接与认证参数

参数名	说明	缺省值	取值 / 约束
`mqtt.enable`	是否启用 MQTT 接入	`false`	`true` / `false`
`mqtt.host`	平台 MQTT 服务端地址	空（必须配置）	域名或 IP
`mqtt.port`	平台 MQTT 服务端端口	`1883`	1 ~ 65535
`mqtt.client_id`	客户端标识	设备序列号（SN）	1 ~ 64 字符
`mqtt.keep_alive`	心跳间隔（秒）	`60`	10 ~ 600
`mqtt.qos`	发布与订阅使用的 QoS	`1`	`0` / `1`
`auth.type`	认证方式	`access_token`	`access_token` / `basic` / `provision`
`auth.access_token`	访问令牌	空	由平台分配
`auth.username` / `auth.password`	用户名 / 密码	空	由平台分配

4.2 Topic 参数

设备使用的每个 Topic 都可配置。如果平台有自己的 Topic 命名规范，可通知安装人员修改下表参数，平台按修改后的 Topic 处理即可。

参数名	用途	设备动作	缺省值
`topic.telemetry`	遥测上报	发布	`v1/devices/me/telemetry`
`topic.attributes_pub`	设备属性上报	发布	`v1/devices/me/attributes`
`topic.attributes_sub`	接收配置属性推送	订阅	`v1/devices/me/attributes`
`topic.attributes_request`	查询属性	发布	`v1/devices/me/attributes/request/{request_id}`
`topic.attributes_response`	接收属性查询结果	订阅	`v1/devices/me/attributes/response/+`
`topic.provision_request`	自动注册请求	发布	`/provision/request`
`topic.provision_response`	接收自动注册结果	订阅	`/provision/response`
`topic.gw_connect`	子设备上线（网关）	发布	`v1/gateway/connect`
`topic.gw_disconnect`	子设备离线（网关）	发布	`v1/gateway/disconnect`
`topic.gw_telemetry`	子设备遥测（网关）	发布	`v1/gateway/telemetry`
`topic.gw_attributes`	子设备属性上报 / 接收推送（网关）	发布 / 订阅	`v1/gateway/attributes`
`topic.gw_attributes_request`	查询子设备属性（网关）	发布	`v1/gateway/attributes/request`

参数名	用途	设备动作	缺省值
`topic.gw_attributes_response`	接收子设备属性查询结果（网关）	订阅	`v1/gateway/attributes/response`

自定义规则：

1. 设备发布的 Topic 不含通配符；设备订阅的应答类 Topic 最后一级为 `+`。
2. Topic 中可使用占位符 `{client_id}`、`{sn}`（设备序列号）、`{request_id}`，由设备运行时替换。例如 `topic.telemetry` 可改为 `acme/plant1/{sn}/telemetry`。
3. `topic.attributes_request` 与 `topic.attributes_response` 需成对修改：设备以请求 Topic 的最后一级作为 `request_id`，并期望在应答 Topic 的同一级收到相同的值。

---

5. 连接与认证

设备在 CONNECT 报文中按 `auth.type` 填写凭据：

`auth.type`	Client ID	Username	Password	平台识别设备的依据
`access_token`（缺省）	`mqtt.client_id`	访问令牌	空	Username 中的令牌
`basic`	`mqtt.client_id`	`auth.username`	`auth.password`	用户名 + 密码（可结合 Client ID）
`provision`	`mqtt.client_id`	`provision`	空	仅用于自动注册，见第 10 章

平台要求：

- 每台设备使用独立凭据，平台根据凭据确定"当前连接是哪台设备"。
- 认证成功返回 CONNACK `0`；凭据错误返回 `4`，设备被禁用或未授权返回 `5`（MQTT 5.0 对应 `0x86` / `0x87`）。
- 设备收到 `4` / `5` 后按最大重连间隔慢速重试，不会高频冲击服务端。

---

6. Topic 定义

缺省 Topic 中的 `me` 表示"当前连接的这台设备"。设备身份由连接凭据确定，Topic 中不携带设备标识，平台应根据连接会话将消息归属到对应设备。

6.1 设备发布（平台接收）

缺省 Topic	内容	平台处理
`v1/devices/me/telemetry`	遥测	解析并按时间序列存储（第 7 章）
`v1/devices/me/attributes`	设备属性	更新设备属性最新值（8.1）
`v1/devices/me/attributes/request/{request_id}`	属性查询	向对应应答 Topic 返回结果（8.3）

6.2 设备订阅（平台发送）

缺省 Topic	内容	平台何时发送
`v1/devices/me/attributes`	配置属性推送	配置属性被修改或删除时（8.2）
`v1/devices/me/attributes/response/{request_id}`	属性查询结果	收到属性查询后（8.3）

注意：`v1/devices/me/attributes` 是双向 Topic。设备向它发布设备属性，同时订阅它接收配置属性推送。平台推送时只发往该设备自己的连接，不能广播给其他设备。

网关与自动注册使用的 Topic 见第 9、10 章。

---

7. 遥测数据

Topic：`topic.telemetry`（缺省 `v1/devices/me/telemetry`）

报文为 UTF-8 编码的 JSON，平台需支持以下三种格式：

格式一：键值对（不带时间戳）。平台以收到消息的时间作为数据时间。

```
{ "temperature": 25.6, "humidity": 68, "running": true, "mode": "auto" }
```

格式二：带时间戳。`ts` 为 Unix 时间戳，单位毫秒，UTC。

```
{ "ts": 1792459200000, "values": { "temperature": 25.6, "humidity": 68 } }
```

格式三：数组（批量上报、断线补传）。数组元素为格式二的对象；补传时每条都带原始采集时间。

```
[
  { "ts": 1792459200000, "values": { "temperature": 25.6 } },
  { "ts": 1792459260000, "values": { "temperature": 25.8 } }
]
```

平台处理要点：

- 值的类型为字符串、数值、布尔或 JSON 对象，平台按 JSON 类型存储。
- 补传数据的 `ts` 可能早于当前时间数小时甚至数天，平台应按 `ts` 存入历史，而不是当作实时值。
- 设备使用 QoS 1，网络异常时同一条数据可能重复到达，平台应按"设备 + `ts` + 键"去重。
- 遇到无法识别的字段应忽略，不要拒收整条消息。

---

8. 属性

8.1 设备属性上报

Topic：`topic.attributes_pub`（缺省 `v1/devices/me/attributes`）

设备每次连接成功后上报一次，之后在属性变化时上报。平台用报文中的键值覆盖该设备属性的最新值。

```
{ "serial_number": "SN20260001", "model": "XX-100", "firmware_version": "1.2.3" }
```

8.2 配置属性推送

Topic：`topic.attributes_sub`（缺省 `v1/devices/me/attributes`）

平台修改某台设备的配置属性后，应立即向该设备推送，报文只包含变化的键：

```
{ "report_interval": 30, "temp_high_threshold": 45.0 }
```

平台删除配置属性时，推送 `deleted` 数组，设备收到后将该项恢复为本地缺省值：

```
{ "deleted": ["temp_high_threshold"] }
```

设备离线时平台无需缓存推送，设备上线后会通过 8.3 主动查询最新值。各型号设备支持的配置属性见设备专用文档。

8.3 属性查询

设备上线后会主动查询配置属性，以同步离线期间的变化。

请求：设备发布到 `topic.attributes_request`，例如 `v1/devices/me/attributes/request/7`

```
{ "clientKeys": "firmware_version", "sharedKeys": "report_interval,temp_high_threshold" }
```

字段	说明
`clientKeys`	逗号分隔的设备属性键，可省略
`sharedKeys`	逗号分隔的配置属性键，可省略

两个字段都省略时，平台返回该设备的全部属性。

应答：平台发布到 `topic.attributes_response` 对应的 Topic，最后一级必须与请求相同，例如 `v1/devices/me/attributes/response/7`

```
{
  "client": { "firmware_version": "1.2.3" },
  "shared": { "report_interval": 30, "temp_high_threshold": 45.0 }
}
```

平台上不存在的键在应答中省略；某一类都不存在时可省略整个 `client` 或 `shared` 对象。

---

9. 网关设备

网关设备以自身的凭据建立一个 MQTT 连接，代理下挂的多个子设备（如 Modbus、BLE 下挂设备）。子设备不单独建立连接，通过报文中的子设备名称区分。子设备名称由网关生成，通常为"网关 SN + 子设备地址"或子设备自身 SN。

平台需维护"网关 → 子设备"的关系：首次收到某个子设备名称时创建该子设备，并将其归属于当前网关。

9.1 网关发布（平台接收）

缺省 Topic	内容	报文示例
`v1/gateway/connect`	子设备上线	`{"device": "GW001-mb-01", "type": "temperature-sensor"}`
`v1/gateway/disconnect`	子设备离线	`{"device": "GW001-mb-01"}`
`v1/gateway/telemetry`	子设备遥测	见下
`v1/gateway/attributes`	子设备属性	`{"GW001-mb-01": {"firmware_version": "1.0.4"}}`
`v1/gateway/attributes/request`	查询子设备属性	见 9.3

- `type` 为子设备类型，可选，平台可据此为子设备套用对应的数据模型。
- 子设备遥测按子设备名称分组，每组为第 7 章格式三的数组；一条消息可包含多个子设备：

```
{
  "GW001-mb-01": [ { "ts": 1792459200000, "values": { "temperature": 25.6 } } ],
  "GW001-mb-02": [ { "values": { "temperature": 31.2 } } ]
}
```

- 子设备在线状态以网关上报为准；网关自身断开连接时，平台应将其所有子设备视为离线。

9.2 网关订阅（平台发送）

缺省 Topic	内容	报文示例
`v1/gateway/attributes`	子设备配置属性推送	`{"device": "GW001-mb-01", "data": {"report_interval": 30}}`
`v1/gateway/attributes/response`	子设备属性查询结果	见 9.3

`data` 的格式与 8.2 相同，也可以是 `{"deleted": [...]}`。平台只需推送已通过 `connect` 上线的子设备。

9.3 子设备属性查询

请求（网关发布到 `v1/gateway/attributes/request`）：

```
{ "id": 1, "device": "GW001-mb-01", "client": false, "keys": ["report_interval", "temp_high_threshold"] }
```

字段	说明
`id`	请求序号，平台在应答中原样返回
`device`	子设备名称
`client`	`true` 查询设备属性，`false` 查询配置属性
`keys` / `key`	多个键用 `keys` 数组，单个键用 `key` 字符串

应答（平台发布到 `v1/gateway/attributes/response`）：

```
// 对应 keys 请求
{ "id": 1, "device": "GW001-mb-01", "values": { "report_interval": 30, "temp_high_threshold": 45.0 } }

// 对应 key 请求
{ "id": 2, "device": "GW001-mb-01", "value": 30 }
```

---

10. 设备自动注册

用于批量部署：平台无需预先逐台建档，设备首次上线时用一组注册密钥向平台申请自己的凭据。

10.1 平台需提供

项目	说明	对应设备参数
注册 Key	标识一批设备或一类产品	`provision.key`
注册 Secret	校验设备合法性的预共享密钥	`provision.secret`

设备名称参数 `provision.device_name` 缺省为设备序列号。

10.2 交互流程

```
设备                                                平台
 | CONNECT  Username = "provision", Password 为空    |
 | SUBSCRIBE /provision/response                     |
 | PUBLISH   /provision/request   ──────────────►    | 校验 Key / Secret，创建设备，生成凭据
 |                                ◄──────────────    | PUBLISH /provision/response
 | 保存凭据 → 断开 → 用新凭据重新连接（第 5 章）      |
```

平台对 Username 为 `provision` 的连接只应允许注册操作。

请求：

```
{ "deviceName": "SN20260001", "provisionDeviceKey": "<注册 Key>", "provisionDeviceSecret": "<注册 Secret>" }
```

字段	说明
`deviceName`	设备名称
`provisionDeviceKey` / `provisionDeviceSecret`	注册 Key / Secret
`credentialsType`	可省略，缺省为 `ACCESS_TOKEN`；为 `MQTT_BASIC` 时设备同时提供 `clientId`、`username`、`password`

应答：

```
// 成功，签发访问令牌
{ "status": "SUCCESS", "credentialsType": "ACCESS_TOKEN", "credentialsValue": "a1B2c3D4e5F6g7H8" }

// 成功，用户名 / 密码方式
{ "status": "SUCCESS", "credentialsType": "MQTT_BASIC",
  "credentialsValue": { "clientId": "SN20260001", "userName": "u-sn20260001", "password": "p@ss" } }

// 失败
{ "status": "NOT_FOUND", "errorMsg": "Provision data was not found!" }
```

status	含义	设备行为
SUCCESS	注册成功	保存凭据并重新连接
NOT_FOUND	注册 Key / Secret 无效或注册未开放	停止注册，等待人工处理
FAILURE	其他失败，如设备名称已存在	按最大重连间隔重试

11. 设备上线时序与连接行为

11.1 上线时序

```
设备                                          平台
 | CONNECT                                    |
 | ------------------------------------------>| 认证
 |                                            |
 |<------------------------------------------ | CONNACK 0
 | SUBSCRIBE topic.attributes_sub             |
 | SUBSCRIBE topic.attributes_response        |
 | ------------------------------------------>|
 |                                            |
 |<------------------------------------------ | SUBACK
 | PUBLISH topic.attributes_pub    设备属性   |
 | ------------------------------------------>| 更新设备属性
 | PUBLISH topic.attributes_request 查询      |
 | ------------------------------------------>|
 |                                            |
 |<------------------------------------------ | 属性查询结果
 | PUBLISH topic.telemetry  补传缓存数据      |
 | ------------------------------------------>| 按 ts 存储
 | PUBLISH topic.telemetry  周期上报          |
 | ------------------------------------------>|
 |                                            |
 |<------------------------------------------ | 配置变化时推送
```

网关在 SUBACK 之后，会先对每个子设备发送 `connect`，再上报子设备数据。

11.2 断线与重连

- 平台在 1.5 × `mqtt.keep_alive`（缺省 90 秒）内未收到设备任何报文，即可判定设备离线。
- 设备断线后按指数退避重连，等待时间依次约为 1、2、4、8、16、32、60、60……秒（含 ±20% 随机抖动）。平台故障恢复后，设备重连会自然分散，不会同时涌入。
- 设备每次重连都会重新订阅、重新上报设备属性并查询配置属性，平台无需保留会话状态。

12. 对接自检清单

序号	检查项	通过标准
1	设备认证	设备连接后 CONNACK 返回 0，平台显示设备在线
2	遥测	三种格式（第 7 章）均能正确解析和存储
3	历史补传	断网一段时间后恢复，补传数据按原始 `ts` 出现在历史中
4	设备属性	设备重连后平台显示最新固件版本等属性
5	配置推送	在平台修改配置属性，设备立即生效
6	配置删除	在平台删除配置属性，设备恢复缺省值
7	属性查询	设备离线期间修改配置，设备上线后生效
8	离线判定	断开设备电源，平台在约 90 秒内显示离线
9	网关	子设备自动创建、数据正确归属、网关断开后子设备显示离线
10	自动注册	使用注册密钥的新设备能获得凭据并正常上线
11	自定义 Topic	如使用了自定义 Topic，以上各项均按新 Topic 通过

13. 常见问题排查

现象	可能原因 / 排查方法
CONNACK 返回 4 / 5	平台凭据与设备配置不一致（注意多余空格或换行）；设备在平台被禁用
设备连接后立即断开	多台设备使用了相同 Client ID；设备订阅的 Topic 未被平台授权
设备在线但平台无数据	平台未按设备实际使用的 Topic 处理；用 MQTT 调试工具订阅该 Topic 查看原始报文
数据时间错误	平台忽略了 `ts`，或把毫秒当作秒处理
设备收不到配置	平台推送的 Topic 与设备 `topic.attributes_sub` 不一致
设备查询属性无响应	应答 Topic 最后一级与请求的 `request_id` 不一致
子设备数据缺失	平台未处理 `v1/gateway/connect`，或未按子设备名称分组解析
报文被截断或连接断开	平台报文上限小于设备补传报文大小，需调大平台上限

14. 附录

14.1 使用命令行模拟设备联调

在真实设备到位前，平台可用通用 MQTT 客户端（如 `mosquitto_pub` / `mosquitto_sub`、MQTTX）模拟设备验证接入逻辑。以下 `mqtt.example.com` 代表平台服务端地址。

```
# 1. 以设备身份订阅配置推送与查询结果（保持运行）
mosquitto_sub -h mqtt.example.com -p 1883 -i SN20260001 -u <设备令牌> -q 1 -v \
  -t 'v1/devices/me/attributes' -t 'v1/devices/me/attributes/response/+'

# 2. 上报遥测
mosquitto_pub -h mqtt.example.com -p 1883 -i SN20260001-pub -u <设备令牌> -q 1 \
  -t 'v1/devices/me/telemetry' -m '{"temperature":25.6,"humidity":68}'

# 3. 补传历史数据
mosquitto_pub -h mqtt.example.com -p 1883 -i SN20260001-pub -u <设备令牌> -q 1 \
  -t 'v1/devices/me/telemetry' \
  -m '[{"ts":1792459200000,"values":{"temperature":25.6}},{"ts":1792459260000,"values":{"temperature":25.8}}]'

# 4. 上报设备属性
mosquitto_pub -h mqtt.example.com -p 1883 -i SN20260001-pub -u <设备令牌> -q 1 \
  -t 'v1/devices/me/attributes' -m '{"model":"XX-100","firmware_version":"1.2.3"}'

# 5. 查询配置属性（结果出现在第 1 步的窗口中）
mosquitto_pub -h mqtt.example.com -p 1883 -i SN20260001-pub -u <设备令牌> -q 1 \
  -t 'v1/devices/me/attributes/request/1' -m '{"sharedKeys":"report_interval"}'
```

注：第 1 步与其余步骤使用了不同的 Client ID，避免相互踢线。若平台限制每台设备只能有一个连接，请改用支持同一连接收发的工具（如 MQTTX）。

14.2 修订记录

版本	日期	说明
V1.0	2026-10-26	首次发布